

DEPLOYMENT GUIDE & ARCHITECTURE

Complete steps, customizations, and configurations for deploying the Portfolio Project backend on PythonAnywhere and the Vite React frontend on Vercel.

System Architecture Overview

This document details the configuration and deployment procedures for your portfolio application. The project consists of a **Django REST API** backend (configured for PythonAnywhere) and a **React + Vite** SPA frontend (configured for static hosting on Vercel). Follow these steps to ensure secure cross-origin communication, static assets delivery, and error-free operation.

Frontend Hosting Platforms Comparison

Platform	Recommended For	Bandwidth Limit	Key Advantage
Vercel	Best DX & Setup	100 GB / month	Zero-config for Vite. Out-of-the-box CI/CD and deploy
Cloudflare Pages	High-traffic Portfolios	Unlimited	Extremely fast global edge CDN. Safe against DDoS/
Netlify	Alternative option	100 GB / month	Excellent git integrations and built-in form handlers.

1. Backend Deployment (PythonAnywhere)

The backend is built with Django and serves content over a REST API. PythonAnywhere serves it using a WSGI web server, with static/media files delivered directly via native web server mappings.

Step 1.1: Git Clone your Code

Open a **Bash Console** on PythonAnywhere and clone your Git repository:

```
git clone https://github.com/Sjmwax/profile.git
cd profile/backend
```

Step 1.2: Set up Virtual Environment & Dependencies

Create a Python 3.10 virtual environment and install dependencies:

```
mkvirtualenv --python=/usr/bin/python3.10 portfolio-venv
pip install -r requirements.txt
```

Step 1.3: Configure Production Environment Variables (`.env`)

Create a `.env` file inside `/home/simbajaymwax/profile/backend/` and configure production parameters:

```
DJANGO_SECRET_KEY=your-secure-production-secret-key
DEBUG=False
ALLOWED_HOSTS=localhost,127.0.0.1,simbajaymwax.pythonanywhere.com
DATABASE_URL=sqlite:///db.sqlite3

# Enable all origins for public portfolio API (solves Vercel preview domain CORS issues)
CORS_ALLOW_ALL_ORIGINS=True
CSRF_TRUSTED_ORIGINS=https://profile1-sjmwax-s-projects.vercel.app
```

CORS & Dynamic Subdomains Warning

Important CORS configuration: Using `CORS_ALLOW_ALL_ORIGINS=True` is highly recommended for public API deployment. Since Vercel creates new subdomains for each git commit (e.g. profile1-riq6p9l9r-...), hardcoding specific origins in settings.py will block API communication on preview links.

Step 1.4: Database Migrations & Django Admin Setup

Run the database migrations and create a superuser for your management panel:

```
python manage.py migrate
python manage.py createsuperuser
```

Step 1.5: Collect Static Files

Gather all static files (for Django admin layout) into the production folder:

```
python manage.py collectstatic --noinput
```

Step 1.6: Configure PythonAnywhere Web Dashboard

In your PythonAnywhere Web tab, set up a **Manual Configuration** app (Python 3.10) and define the following paths:

- **Source code:** /home/simbajaymwax/profile/backend
- **Working directory:** /home/simbajaymwax/profile/backend
- **Virtualenv:** /home/simbajaymwax/.virtualenvs/portfolio-venv

Step 1.7: Set up WSGI File

Click your ****WSGI configuration file**** link on the Web page and replace its contents to load environment variables from your `.env` file properly:

```
import os
import sys
from pathlib import Path

path = '/home/simbajaymwax/profile/backend'
if path not in sys.path:
    sys.path.append(path)

# Load environment variables from .env file
from decouple import Config, RepositoryEnv
env_path = os.path.join(path, '.env')
if os.path.exists(env_path):
    config = Config(RepositoryEnv(env_path))
    for key in RepositoryEnv(env_path).data.keys():
        os.environ[key] = config(key)

os.environ['DJANGO_SETTINGS_MODULE'] = 'portfolio.settings'

from django.core.wsgi import get_wsgi_application
application = get_wsgi_application()
```

Step 1.8: Static and Media File Mappings

In the ****Static files**** section of the PythonAnywhere Web tab, map the URL patterns to physical server folders. This enables PythonAnywhere to bypass Django and serve CSS/images directly for max performance:

URL Path	Directory Path
/static/	/home/simbajaymwax/profile/backend/staticfiles
/media/	/home/simbajaymwax/profile/backend/media

2. Frontend Deployment (Vercel)

Since the React frontend is compiled to a static Single Page Application via Vite, we can deploy it directly onto Vercel.

Step 2.1: Import Project and Set Root Directory

1. Log in to Vercel and import your GitHub repository.
2. In the configuration settings, locate **Root Directory** and change it from `./` to **frontend**. This tells Vercel where your Vite files are located.
3. Ensure build command is `npm run build` and output folder is `dist`.

Step 2.2: Add Environment Variables

Before building, add the following environment variable to Vercel:

- **Key:** `VITE_API_URL`
- **Value:** `https://simbajaymwax.pythonanywhere.com/api`

Step 2.3: Build & Trigger Redeployment

Build-time vs Run-time Variables

Why redeploy is crucial: Because React runs entirely client-side in the browser, it cannot read environment variables at runtime. Instead, Vite injects `VITE_API_URL` into the Javascript files **during build time**. If you modify or add this variable in Vercel settings, you *must* click **Redeploy** in Vercel so the compilation is refreshed.

3. Critical Troubleshooting & Gotchas

1. Invalid Host Header (Bad Request 400)

If you get a **400 Bad Request** or a `DisallowedHost` error on PythonAnywhere, you must make sure that `simbajaymwax.pythonanywhere.com` is added to `ALLOWED_HOSTS` in your `.env` file. Do not include `https://` in this variable, just the raw domain.

2. Trailing Slashes on CORS Origins

When specifying `CORS_ALLOWED_ORIGINS`, **never** add a trailing slash (e.g. `https://my-site.vercel.app/`). It must be the clean origin: `https://my-site.vercel.app`. Adding a slash will cause the browser to block API requests.

3. Mixed Content Block

Vercel sites run under secure HTTPS. Therefore, your `VITE_API_URL` in Vercel settings must also begin with **https://**. If you use `http://`, the browser will block the API requests as insecure mixed content.

4. Missing Data Placeholder Fallbacks

If your frontend successfully connects to the backend but renders blank sections, it means your database on PythonAnywhere is empty. Log into your Django admin, navigate to **Portfolios**, and click **Add Portfolio**. Once a record exists, your frontend will populate automatically.